

An Empirical Study of Java Code Improvements Based on Stack Overflow Answer Edits

In-on Wiratsin^{1,✉}, Chaiyong Ragkhitwetsagul^{1*,✉}, Matheus Paixao², Denis De Sousa², Pongpop Lapvikai¹, Peter Haddawy¹

¹ Faculty of Information and Communication Technology, Mahidol University, Nakhon Pathom, Thailand

² State University of Ceara (UECE), Fortaleza, Brazil

✉These authors contributed equally to this work.

* chaiyong.rag@mahidol.ac.th

Abstract

Suboptimal code is prevalent in software systems. Developers may write low-quality code for various reasons, including a lack of technical knowledge, insufficient experience, time pressure, management decisions, and even personal unhappiness. Once suboptimal code is integrated into a software system’s codebase, its accumulation can lead to significant maintenance costs and technical debt.

During their day-to-day work, developers consult many external knowledge bases, including Google searches, API documentation, and Q&A websites, to help with their programming tasks. Stack Overflow is a popular Q&A website where programmers can ask questions and share code snippets. Its crowdsourced and collaborative nature has created a vast repository of programming knowledge that can be leveraged to support developers. Under the community-curated content policy, Stack Overflow answers are always evolving, with new answers posted or existing ones edited.

In this paper, we present an empirical study of Stack Overflow Java answer edits and their application to improving code in open-source software projects. To perform the study, we employ a code clone search tool, with modifications, to analyse Stack Overflow code snippets with version history. The tool is applied to open-source Java projects to identify outdated or unoptimised code snippets and suggest optimised alternatives. By analysing 140,840 Java accepted answers from SOTorrent and 10,668 GitHub Java projects, we manually categorised the Stack Overflow answer edits and then created pull requests containing the code edits to open-source projects. Our results show that 6.91% of Stack Overflow Java accepted answers have more than one revision, with the average number of revisions per answer being 2.82. Moreover, 49.24% of the code snippets in the Stack Overflow answer edits are considered applicable to open-source projects, and 11 out of 36 proposed bug fixes based on the answer edits were accepted by the GitHub project maintainers.

1 Introduction

Suboptimal code is prevalent in software systems. Developers may write low-quality code for various reasons, including a lack of technical knowledge [1], limited experience [2], time pressure [3], management decisions [4, 5], and even unhappiness [6]. Regardless of the reason, lack of proper remediation may lead to large maintenance costs [7] and technical debt [8, 9].

Stack Overflow (SO)¹ is a popular website for programmers to ask questions and share their code snippets. The collaborative nature of SO has created a large source of programming knowledge. As a result, programmers commonly reuse code from SO answers in their software projects [10–12]. A recent study by Chen et al. [13] found that 9% of SO answers have their code copied into open-source projects without modification. Other studies report that 6.32% to 8.38% of code in open-source projects is reused from SO [14, 15]. Abdalkareem [16] confirms in a recent study that reusing crowdsourced code aids software development but may also degrade software quality through dependency overhead and increased maintenance effort. Yu et al. [17] further characterise reusable code snippets by their prevalence, fault-resiliency, and longevity, providing criteria for prioritising high-quality reused snippets. With the launch of ChatGPT and other generative AI models, the popularity of SO is declining [18, 19]. Xue et al. [20] report that the availability of large language models reduces SO question volume by 14–28%, but the decline concentrates in routine, well-documented topics, resulting in niche partitioning rather than full displacement. Similarly, Shan et al. [21] find that generative AI changes, rather than eliminates, users’ voluntary knowledge contribution on SO. Indeed, a recent study [22] shows that SO remains useful for developers, as the questions and answers on SO are evolving to address complex and challenging problems that are not handled by such generative AIs.

Although code snippets from SO may contain issues such as security vulnerabilities [23], API misuses [24], and license-violating code [11, 12], the crowdsourcing nature of the website allows for constant community updates that not only optimise the answers’ code but also fix potential issues [25, 26]. A recent study [27] shows that 55.3% of update request comments on SO result in an updated snippet within 24 hours. Moreover, Mondal et al. [28] found that 53.3% of the answer edits improve the answers to match the questions, and 51% of the edits aim to optimise code performance. In a similar vein, Zuo et al. [29] track how security-relevant C# snippets evolve through user revisions, cautioning that the quality of edits depend on the editor’s experience. Jallow et al. [30] further show that SO code snippets are non-stationary over time and recommend treating SO as a time-series data source, which reinforces the value of analysing the version histories of answer edits.

Several studies have proposed approaches that leverage SO knowledge to assist developers. Some of them create automated tools that provide working code examples [31], show relevant SO posts based on the code context in the IDE [32, 33], and improve API documentation [34]. Tang et al. [26] proposed a method to automatically identify comment-edit pairs on SO, i.e., comments in an answer that trigger an edit in the answer, and recommend the edits to projects with the same outdated snippet. Gao et al. [35] propose a technique to retrieve semantically equivalent questions from SO and provide a ranked list of code snippets as recommendations. Jallow et al. [36] examine the adoption of outdated code snippets from SO, focusing on those that contain security issues. They reported 43 popular GitHub projects missing fixes to bugs and vulnerabilities based on the answers on SO.

Nonetheless, a gap remains in these existing studies. First, the recommendations are limited to the code context in the IDE. Second, matches are only made to the latest version of code answers on SO, so that possible matches to earlier versions are missed. Third, the studies focusing on Stack Overflow updates are based on updates triggered by comments, excluding answer edits for other reasons. Fourth, the recommendations were based solely on exact matches between code on SO and in software projects, overlooking code that had been modified. Lastly, they studied only popular projects on GitHub, ignoring the less popular ones that might also benefit from SO code recommendations.

In this paper, we fill the gap by performing an empirical study of SO Java answer

¹<https://stackoverflow.com/>

edits and their recommendations as potential code improvements. We analysed 140,840 Java accepted answers with revisions on SO to identify those that included revisions. Leveraging a scalable code clone search tool with optimised configurations for locating code clones, with exact matches and with modifications, between code snippets in the collected SO accepted answers and 10,668 Java open-source projects. The projects were classified into three tiers based on their popularity. By comparing the outdated code with its newer version (i.e., the proposed update) on SO, we classified the proposed updates into groups based on their update types. Lastly, we investigated whether developers found the proposed updates based on SO answer edits useful by opening pull requests containing those edits. In contrast to prior work, which recommends only the latest version of an answer, relies on comment-triggered edits, matches code exactly, or studies only popular projects, our study is the first to jointly (1) drive recommendations from the full answer edit history, matching project code against earlier revisions; (2) consider all answer edits regardless of what triggered them; (3) match code with modifications rather than exact clones only; and (4) validate the recommendations end-to-end through real pull requests across projects of all popularity levels.

This paper makes the following contributions.

1. A large-scale study of SO answer edits analysing the full history of 140,840 Java accepted answers with revisions and 10,668 Java open-source projects covering both less-popular and highly-popular projects.
2. A specialised clone search tool that supports code revisions on SO, which is made publicly available².
3. A classification of code improvements based on the Stack Overflow answer edits.
4. A dataset of manually validated code clones between SO and GitHub projects that are made publicly available, which can be used for future clone studies.

This empirical study provides insights into the categories of SO Java answer edits and their applications to open-source software projects, and paves the way for automated recommendations for code updates based on SO answer edits.

1.1 Motivating Example

As motivation for this study, we take a look at the SO question number 40665315³. In the question, the developer is using the REST Assured⁴ library to write unit tests for a REST application developed with the Spring Boot⁵ framework. For this particular unit test, the developer needs to specify the port on which the service will run, which is the main topic of the question. A solution was given and was marked as the accepted answer in November 2016.

However, in December 2018, the accepted answer was edited further as shown in Figure 1. We can see that the new `setUp()` method has been added, and the call to method `.port()` in the test has been removed. This update eliminates the need to set up the port every time the test is executed, thereby saving resources.

We conducted an initial search for instances of the outdated code-answer version in this answer on GitHub and found that 7 out of 10 projects are still using this sub-optimal solution. As this example shows, suboptimal solutions can be posted (and accepted) on SO, and it may take the community a while to optimise them. In the meantime, developers may use the sub-optimal snippets in their own solutions. This example

²<https://github.com/cragkhit/Matcha>

³<https://stackoverflow.com/questions/40665315>

⁴<https://rest-assured.io/>

⁵<https://spring.io/projects/spring-boot>

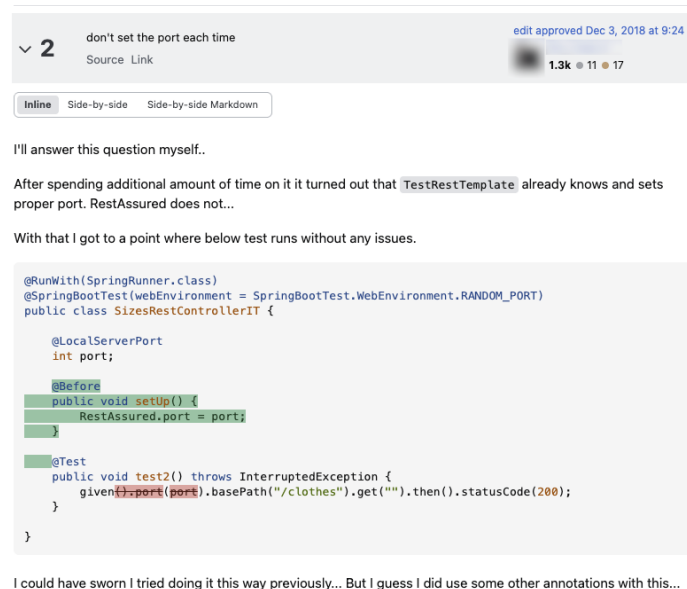


Fig 1. An example of the code answer edit on an SO accepted answer, which optimises the code by moving the port setup to the `@Before` method.

illustrates that software projects that use suboptimal code may benefit from adopting the more recent, optimal code solutions on SO.

1.2 Research Questions

Our study is designed as a pipeline that moves from establishing the supply of Stack Overflow answer edits, to matching those edits against source code in GitHub projects, and finally to validating their usefulness in practice. We first quantify how often accepted Java answers are edited after they are posted, which establishes the pool of potential code improvements available for recommendation (RQ1). We then examine, for code found in open-source projects that is similar to an earlier revision of such an answer, what types of improvements the corresponding answer edits suggest and how these are distributed across projects of differing popularity (RQ2). Finally, to confirm that these matched edits translate into changes that developers actually value, we submit a subset of them as pull requests to the corresponding projects (Section 4, Phase 3). This last step is a practical validation of the pipeline rather than a separately declared research question.

This study seeks to answer the following research questions:

- **RQ1:** *To what extent are accepted Java answers on Stack Overflow edited?* For this RQ, we want to understand the prevalence of edits to SO Java accepted answers. We extract the revision history of SO Java accepted answers to determine how many answers were edited after they were posted and how large those edits are. This establishes the supply of potential code improvements that could later be recommended to developers.
- **RQ2:** *What types of code improvements do Stack Overflow answer edits suggest for code found in open-source projects, and how are they distributed across projects of different popularity?* For this RQ, we use the supply of potential code improvements from RQ1 to to (1) gain deeper understanding of the types of code improvements suggested by SO answer edits, and (2) determine how often these improvements

are applicable to open-source projects. We classify the answer edits into categories based on the types of code improvements they suggest, and we examine how these categories are distributed across projects of differing popularity.

2 Related Work

2.1 Stack Overflow Answer Edits

A few previous studies have examined aspects of SO answer edits. Wang et al. [37] investigate how the SO badge system, which rewards users based on the quantity of revisions, incentivised answer revision behaviour and quality. Analysing over 2,377,692 SO answers, the study reveals that users make significantly more revisions on badge-awarding days than on normal days. These revision spikes are associated with a higher likelihood of the revision being rolled back (i.e., rejected), often due to undesired code/text formatting or incorrect changes. During these spikes, users tend to make small, text-only revisions (e.g., text correction and code formatting) rather than complex code corrections or improvements. Baltes et al. [38] similarly analysed the SO post edits, but based on their edit comments. They reported that 17.5% modified only a code block, 64.2% only a text block, and 18.3% both text and code blocks of a particular SO post. They also categorised the post edits into 25 categories (e.g., Formatting, Code, Fixing, Improving) by manually analysing frequent messages and using regular expressions. Formatting, adding, and fixing were the most frequent categories across all edits, as well as in the subset that modified code blocks only. Specifically addressing non-functional properties, the authors identified a few thousand code edits that mentioned performance, size, and memory. The findings suggest that SO data can serve as a treasure trove for future work on mining fine-grained patches.

Tang et al. [26] follow the same line of research as they investigate the potential of using SO comment-edit pairs as a complementary data source for building code maintenance data sets, which are crucial for tools like program repair and code recommenders. The authors propose an automated technique to extract these pairs, matching a comment suggesting a change to a subsequent code edit and providing concrete explanations for the change. Applying their technique to five popular programming languages (Java, JavaScript, Android, PHP, Python), they find that the extracted edits are rarely tangled (11%) and that 27% of the confirmed pairs are potentially useful for code maintenance applications. The study resulted in 10 pull requests to popular open-source repositories based on the mined pairs. Sheikhaei et al. [27] study the update request comments (URCs), i.e., comments on SO that point out issues like buggy code or obsolete information, that trigger an update to the answer post and improve its quality. Through an empirical analysis of 1,221 comments, they find that although URCs are prevalent (51.7%), only 5.3% are addressed within 24 hours. However, 36.5% remain unaddressed after a year, and the community voting mechanism is ineffective at surfacing these issues.

Zhang et al. [39] focus specifically on C/C++-related answers and revisions on SO and their security weaknesses. They found that a significant 36% of security weaknesses (CWEs) were detected, and while revisions generally reduce these weaknesses, a majority of vulnerable snippets are never corrected, exposing users to risks. Jallow et al. [36] found that 51% of SO answers in open-source projects become outdated because developers fail to track the original source, leading to missed updates, including fixes for up to 15 distinct security issues in 43 popular GitHub projects. For C/C++ code, this includes prevalent flaws such as buffer overflows. They suggest that tools are urgently needed to monitor SO for fixes and effectively surface the best, most current code solutions.

Mondal et al. [28] investigate whether SO answer edits enhance the answer quality. Analysing 94,994 edited Python answers on SO, they compare the initial and latest

versions across six key dimensions: semantic relevance, code usability, complexity, security, optimisation, and readability. The findings show that the answer edits offer both benefits and drawbacks. The benefits include improving semantic relevance in 53.3% of cases and making 9% of previously broken code executable. On the other hand, there are negative impacts, such as increased code complexity (32.3%), decreased readability (49.7%), and introducing new security vulnerabilities in 20.5% of cases rather than fixing existing ones.

2.2 Generating Code Recommendations

Previous work has explored basic approaches to generating code recommendations from SO data. The work of Ponzanelli et al. [32, 33] presents PROMPTER, an IDE plugin that recommends Stack Overflow discussions based on the developer's current coding context. By leveraging existing search engines, PROMPTER identifies relevant discussions based on a predefined threshold. A developer working in an IDE can open and visualise the Stack Overflow discussion directly in the IDE. Although it does not directly recommend code snippets to developers, PROMPTER leverages Stack Overflow's knowledge to enhance the developer experience in the IDE by providing more context for their coding decisions. On a similar note, the work by Rahman et al. [40] proposes RACK, an automated tool that focuses on API recommendations based on Stack Overflow knowledge. Unlike the previously mentioned work, the RACK tool utilises natural language queries to search for relevant Stack Overflow posts. The work by Rubei et al. [41] aims to recommend relevant posts based on a developer's coding context. The proposed tool, called POSTFINDER, focuses on enhancing both Stack Overflow posts and developers' code with additional metadata to improve matching. The results demonstrate that the new features facilitate matching code context with highly relevant posts. Gao et al. [35] propose QUE2CODE, a two-stage, query-driven model consisting of semantically equivalent question retrieval and best code snippet recommendation, designed to recommend the best code snippets from Stack Overflow posts to address developer challenges of query mismatch and information overload.

The community working on code recommendations has been rapidly growing, with papers proposing creative ways to recommend code from sources other than Stack Overflow. Keivanloo et al. [31] propose an approach to find working code examples from a large code corpus on the Internet by using maximal frequent itemset mining and a custom search ranking function. Nyamawe et al. [42] recommend refactoring solutions by using traceability and code metrics. CODER [43] is a graph-based code recommendation framework that considers both user-code and user-project interactions to enhance recommendations. Aroma [44] is a tool that provides code recommendations via a structural code search. Guo et al. [45] explore the potential of ChatGPT in refining existing code based on human-written code reviews.

3 Methodology

This section describes the methodology we employed to answer each of our [two](#) research questions, including the dataset, the code clone search tool, and the experimental framework.

3.1 Datasets

The SO data for this study were retrieved from the SOTorrent [46] dataset, the largest SO dataset to date. The dataset is created from the SO data dump, augmented with the version history of SO content. The version history can be retrieved at the level of the

whole post or individual post block. The SOTorrent dataset has been widely used by several previous studies [38, 47–49]. In its latest version (version 2020-12-31), the dataset contains the content of 51,296,931 SO posts with 81,536,422 post versions. The dataset can be accessed via Google BigQuery⁶ or Zenodo⁷.

To search for GitHub projects, we employed GHS (GitHub Search) [50], a tool and dataset tailored for mining software repositories, specifically for searching GitHub projects. The GHS dataset comprises 735,669 repositories written in 10 programming languages. For each project, GHS provides data on 25 characteristics that can be used as filters in the search. For this exploratory study, we searched GHS with the following filters: `Language: Java; Exclude Forks; Has Open Issues; Has Open Pull Requests`. We only selected projects written in `Java` because it is one of the most widely used programming languages [51] and is supported by our selected code clone search tool (see Section 3.2). We excluded all forks to ensure our dataset contained no redundant projects, and selected projects with both open issues and open pull requests to ensure their aliveness.

3.2 Code Clone Search Tool

To locate adoption of Stack Overflow code snippets in open-source projects, we chose Siamese [11] for this study. Siamese is a code clone search tool that is both accurate and scalable, capable of handling hundreds of millions of lines of code. The tool encompasses multiple code representations by transforming code into various forms to capture different types of code clones, query reduction that retains only highly relevant terms in the code query, and a customised ranking function that enables the selection of a preferred clone type to be ranked first. It offers an accurate clone search with high precision and recall for Type-1 (i.e., code clone with exact copies) to Type-3 clones (i.e., code clones with modified/added/removed statements), outperforming other state-of-the-art code search and code clone detection tools [52]. Although numerous code clone detection and search tools exist, only a few are scalable to large-scale code bases with millions of lines of code. A scalability comparison with six existing tools reveals that only Siamese and SourcererCC [53] can scale to a code corpus of 365 million lines of code; however, Siamese can return clone search results in just 8 seconds.

3.3 Experimental Design

Figure 2 shows the experimental design of our study. The study was separated into four sequential phases, described in the subsections below. All the code and data necessary to replicate this study are available in our replication package⁸. This section provides a detailed description of each phase of our study.

3.3.1 Phase 1: Optimising Configuration and Modifying the Code Clone Search Tool

This phase is a preparation for the experiment. We first used a search-based technique to determine the optimised configurations of Siamese. We then augmented Siamese to detect SO’s outdated code in software projects and enable it to provide the latest code answer based on answer edits. We explain these steps in detail below. We call this modified version “Siamese+” and use it for the subsequent phases of the study.

⁶<https://console.cloud.google.com/bigquery?project=sotorrent-org>

⁷<https://zenodo.org/record/3746061>

⁸<https://doi.org/10.5281/zenodo.17220746>

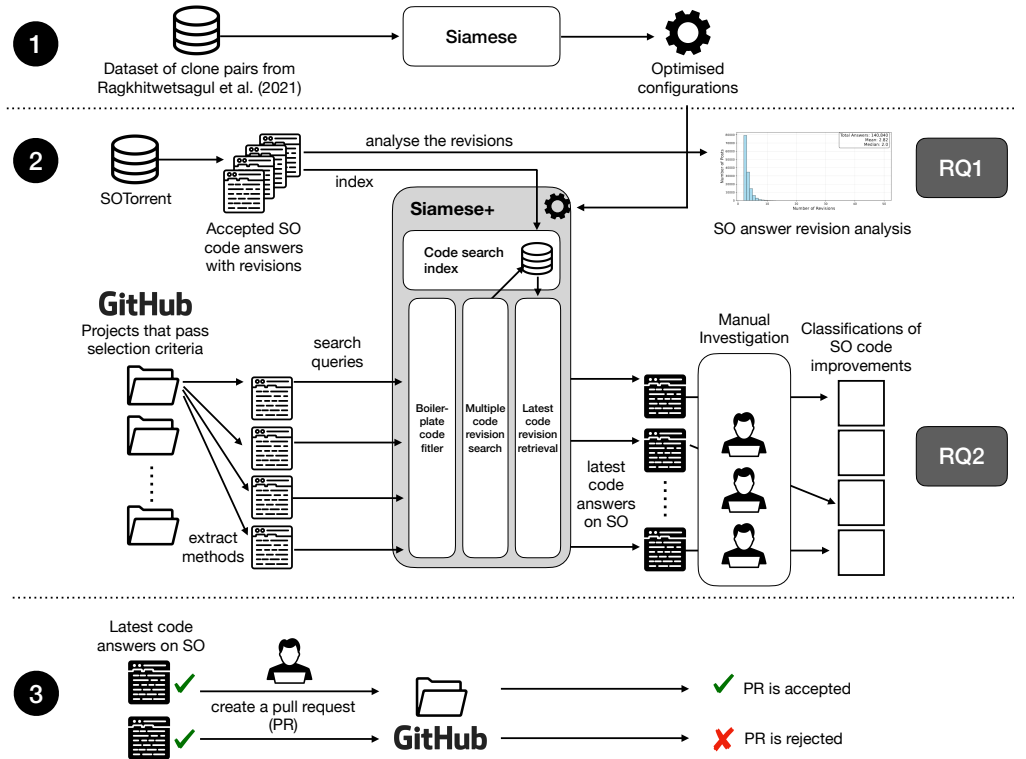


Fig 2. Overview of our three-step exploratory study. Step 1 tunes the Siamese clone search tool, Step 2 detects clones between Stack Overflow answer revisions and GitHub projects (answering RQ1 and RQ2), Step 3 submits the bug-fixing answer edits as pull requests.

Step 1 Optimising Siamese Configurations: A previous study shows that the performance of code similarity analysers depends heavily on the dataset and its configurations, and that reusing default configurations does not always yield optimal detection accuracy [54]. Hence, our first required step is to assess how well Siamese performs at locating similar code across SO answers and Java software projects. Siamese has not yet been directly evaluated regarding its accuracy in matching code from GitHub projects and SO snippets. Moreover, previous work [55] shows that clones between SO and GitHub projects may differ from regular clones, including incomplete code snippets and more boilerplate code. Thus, it is essential to evaluate Siamese configurations to determine the optimal parameter settings for this task.

To perform the configuration optimisation, we leveraged the ground truth dataset⁹ introduced by a previous study [56]. This dataset contains 2,302 pairs of online code clones identified between SO and Java projects in the Qualitas corpus [57], a curated collection of 111 Java projects spanning across different domains. These clone pairs were initially detected by SourcererCC [53] and Simian¹⁰, and then manually reviewed by the authors. The identified clones were categorized into seven distinct patterns of code cloning: Cloned from Qualitas project to Stack Overflow (QS), Cloned from Stack Overflow to Qualitas project (SQ), Cloned from an external source to Stack Overflow (EX), Cloned from each other or from an external source outside the project (UD), Boilerplate or IDE auto-generated code (BP), Inheritance or interface implementation (IN), and Not clones (NC). For this study, only the clone pairs that could be reused between real Java projects and Stack Overflow were selected. So, we filtered the clone ground truth to retain only code clones classified as QS, EX, and UD. This refinement resulted in a final dataset comprising 553 clone pairs.

The parameter tuning approach focused on maximising the mean reciprocal rank (MRR) metric. MRR was chosen because it aligns with the study’s goal: finding the best single match between SO answers and code in Java software projects. The MRR metric is defined as shown in Equation 1, where Q represents the set of all code snippets (queries) in the ground truth, each snippet is denoted as $q \in Q$, and the Reciprocal Rank (RR) is calculated using Equation 2.

$$MRR(Q) = \frac{\sum_{q=1}^{|Q|} RR(q)}{|Q|} \quad (1)$$

$$RR(q) = \frac{1}{rank(S(q))}, \quad \text{if } |S(q)| > 0$$

$$RR(q) = 0, \quad \text{if } |S(q)| = \emptyset \quad (2)$$

To assess Siamese using the ground truth, we indexed the code snippets extracted from the Java projects in the Qualitas corpus and performed searches using code snippets from SO. The SO code snippets were obtained via StackExchange¹¹. An SO post may include multiple code blocks, each potentially containing several lines of code. Thus, for each post listed in the ground truth, we extracted the relevant code snippet based on the specified start and end lines. In cases where a single post contained multiple snippets forming clone pairs, we extracted each snippet individually to maintain their distinction. As part of this extraction process, we also updated the ground truth to reflect the precise start and end lines of Stack Overflow code snippets.

The Java projects from the Qualitas benchmark were obtained from the official website¹². For this study, we retrieved the release version 20130901r, which matches

⁹<https://ucl-crest.github.io/cloverflow-web/downloads.html>

¹⁰<https://simian.quandarypeak.com/>

¹¹<https://archive.org/details/stackexchange>

¹²<http://qualitascorpus.com/download/>

Table 1. Siamese parameters considered in this study.

Parameter	Values
n-gram size	4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24
QR thresholds	2, 4, 6, 8, 10, 12, 14, 16, 18, 20
boosting	-1, 1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20
minCloneSize	6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
simThreshold	{10%,20%,30%,40%}, {20%,30%,40%,50%}, {30%,40%,50%,60%}, {40%,50%,60%,70%}, {50%,60%,70%,80%}, {60%,70%,80%,90%}, {10%,30%,40%,50%}, {20%,40%,60%,70%}, {30%,60%,80%,90%}

the one used in constructing the ground truth [56]. In total, the Qualitas benchmark used in this empirical analysis comprises 111 projects, encompassing 166,709 Java files and 19,614,083 lines of code. The only preprocessing operations applied to the Qualitas source code involved the removal of code comments and the use of pretty printing¹³ to standardise the code structure.

We employed Grid Search as our search algorithm to optimise the Siamese configurations. Despite its simplicity, it avoids the common pitfalls reported in the literature regarding clone detector tuning [58, 59]. Table 1 lists the Siamese parameters we considered. We selected the same parameters used in the original Siamese publication [11] and expanded by considering a larger set of values for each parameter. Then, we selected a pre-defined set of values in the search space (a grid). The chosen values included the n-gram size values of {4, 6, 8}, QR threshold values of {8, 10}, boosting values of {-1, 10}, minCloneSize values of {6, 10} and two sets of simThreshold values of {20%,40%,60%,80%} and {30%,50%,70%,90%}. The Grid Search algorithm then performed an exhaustive search over the grid-defined search space.

Step 2 Modifications of Code Clone Search Tool: To support the goal of this study, we augmented Siamese to include three additional modules: (1) boiler-plate code filter, (2) multiple-code revision search, and (3) latest code revision retrieval (as shown in Figure 2). The three modules are crucial for analysing code snippets in the revisions of Stack Overflow answers and locating the latest version. Although the input is provided to Siamese as a project, the tool reads each Java file in the project, parses it, and performs the code clone search at the method level, one at a time. Within the search process, we incorporated the three additional modules to enable Siamese to return code recommendations as follows.

Boilerplate Code Filter: Clones between Stack Overflow and software projects often contain numerous boilerplate code snippets (e.g., getters, setters, or `equals` methods) that are not useful for developers. The boilerplate code filter removes such boilerplate code from the search. The filter was created as follows. First, we compiled a list of code patterns considered boilerplate according to the classification in a previous study [55]. For example, the `getter`, `setter`, `equals()`, `compareTo()`, `toString()` methods are considered boiler-plate code. Second, we created a list of regular expressions that match such boilerplate code patterns. We integrated these regular expressions into the Siamese search component as a query filter. A certain code query is checked against the list of regular expressions. If a match is found, the code snippet is excluded from the search, thereby removing boilerplate code from the results and avoiding trivial recommendations to developers.

Multiple Code Revision Search: This module allows Siamese to search for multiple revisions of the same code snippet in a SO answer. The module was created as follows. First, we created a clone search index for Siamese by inputting all revisions of each Stack Overflow accepted answer in Java. We named the code snippets in each revision by concatenating the `PostID` (the unique ID of each answer), the `LocalID` (the

¹³<https://astyle.sourceforge.net>

unique ID of each code block within the code), and the **HistoryID** (the unique ID of each revision of the code block) as defined by the SOTorrent dataset. As an example, take the SO answer ID 8394534, which has 2 revisions¹⁴. Each revision was indexed as follows. The code snippet in the original version was saved in a file and indexed under the name `8394534_0_original.java`. The first revision version was created and indexed as `8394534_0_1.java`. The second revision, the latest one, will be created and indexed as `8394534_0_latest.java`. Using this naming technique, we can automatically determine from the Siamese clone search results whether the matched code snippet is the latest version by checking the file name in the first-ranked result.

Latest Code Revision Retrieval: This module allows Siamese to return the latest revision of the matched code snippet. Based on the results from the multiple code revision search module, Siamese checks whether the latest version of the SO answer is returned for each code snippet. If not, Siamese includes the latest version of the answer in the list of code recommendations. After completing the search using all the methods in the given Java project, Siamese returns a list of code recommendations in CSV format. Each record contains the file name, method name, start line, and end line of code in the project, as well as the **PostID** of the Stack Overflow post containing the latest code revision.

3.3.2 Phase 2: Detecting Clones Between Stack Overflow and GitHub

In Phase 2, we conducted a code clone search across SO’s Java accepted answers and GitHub’s Java projects.

Indexing Stack Overflow Java accepted answers with all revisions: Our analysis of Java posts on SOTorrent data shows that there are 2,038,242 Java accepted answers on Stack Overflow. From these accepted answers, there were 140,840 Java accepted answers that had at least one revision. First, we extracted code snippets with revisions from all accepted Java answers. Next, we extracted 283,838 code snippets of all the revisions of the Java-accepted answers and imported them into the Siamese+ clone search index. This indexing phase occurs only once in the study, and subsequent use of Siamese+ will be via a query to a similar code snippet, which takes significantly less time (e.g., seconds).

Filtering of GitHub Projects: As previously mentioned, one of our assumptions for this study is that SO answer edits may provide optimised code snippets that could improve software project codebases. In this context, it is expected that projects with different levels of popularity may benefit differently, as the popularity may serve as a proxy for their code quality. For instance, a project with a high-popularity codebase may not have as many sub-optimal snippets, where Siamese+ would not find as many useful recommendations. In contrast, a project with a low-popularity codebase may contain more suboptimal snippets and might be greatly impacted by Siamese+’s recommendations. To evaluate this assumption, we needed a representative sample of software projects with different levels of code quality.

For each project, GHS provides 3 popularity metrics: **Number of Stars**, **Number of Watchers**, and **Number of Forks**. GitHub popularity metrics, such as those provided by GHS, have been used as quality proxies for GitHub projects [60–62]. Although isolated popularity metrics, such as stars, have been shown not to be the most effective way to assess a project’s code quality [63], we believe that combining these metrics could yield a trustworthy proxy. Hence, for each popularity metric provided by GHS, we computed the distribution and divided it into quartiles. Projects that appeared above the third

¹⁴<https://stackoverflow.com/posts/8394534/revisions>

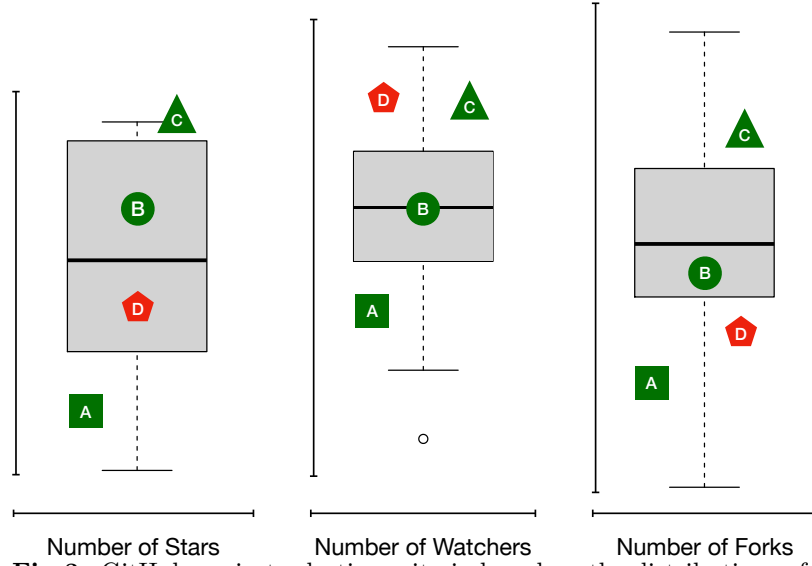


Fig 3. GitHub project selection criteria based on the distributions of a number of stars, watchers, and forks.

quartile for all metrics were considered to have a high-popularity codebase. Similarly, projects that appeared below the first quartile for all metrics were considered to have a low-popularity codebase. Finally, a project that fell between the first and third quartiles for all metrics was considered to have a medium-popularity codebase. Projects that appear in the intersection of quartiles between metrics, e.g., above the third quartile for a metric and below the first quartile for another metric, were excluded from this study. These criteria provided a clear separation of projects based on the proxy we employed for the popularity metrics.

Figure 3 depicts our project selection criteria. Suppose the boxplots represent the distribution of the number of stars, number of watchers, and number of forks for all the projects that pass our GHS filter. There are four projects represented by four symbols: Project A (square), Project B (circle), Project C (triangle), and Project D (pentagon). According to our project selection criteria, we include Project A since it appears below the first quartile for all the metrics. Similarly, we include Projects B and C, as they fall between the first and third quartiles and above the third quartile for all metrics. We do not include Project D because it appears in a different quartile for at least two metrics.

Searching for Clones Over Revisions: After selecting the projects, we ran Siamese+ for each project. First, for a given software project, Siamese+ extracts the code for all the project’s methods and applies the boilerplate code filter. Next, for each remaining method, Siamese+ searches its index, which contains the code for SO’s accepted answers, including all revisions. If Siamese+ locates a snippet from an older revision of a Stack Overflow answer (i.e., not the latest), it returns the latest version of the answer as a recommendation.

Manual Classifications of Potential Clones: After running Siamese+ for all selected projects and collecting all recommendations, we entered the final step of Phase 2. For each potential clone, we looked at the original SO answer, the latest SO answer, the GitHub project code and performed a manual classification. We initially adopted the SO post-edit categorisation proposed by [38] (see Table 2). Each category describes

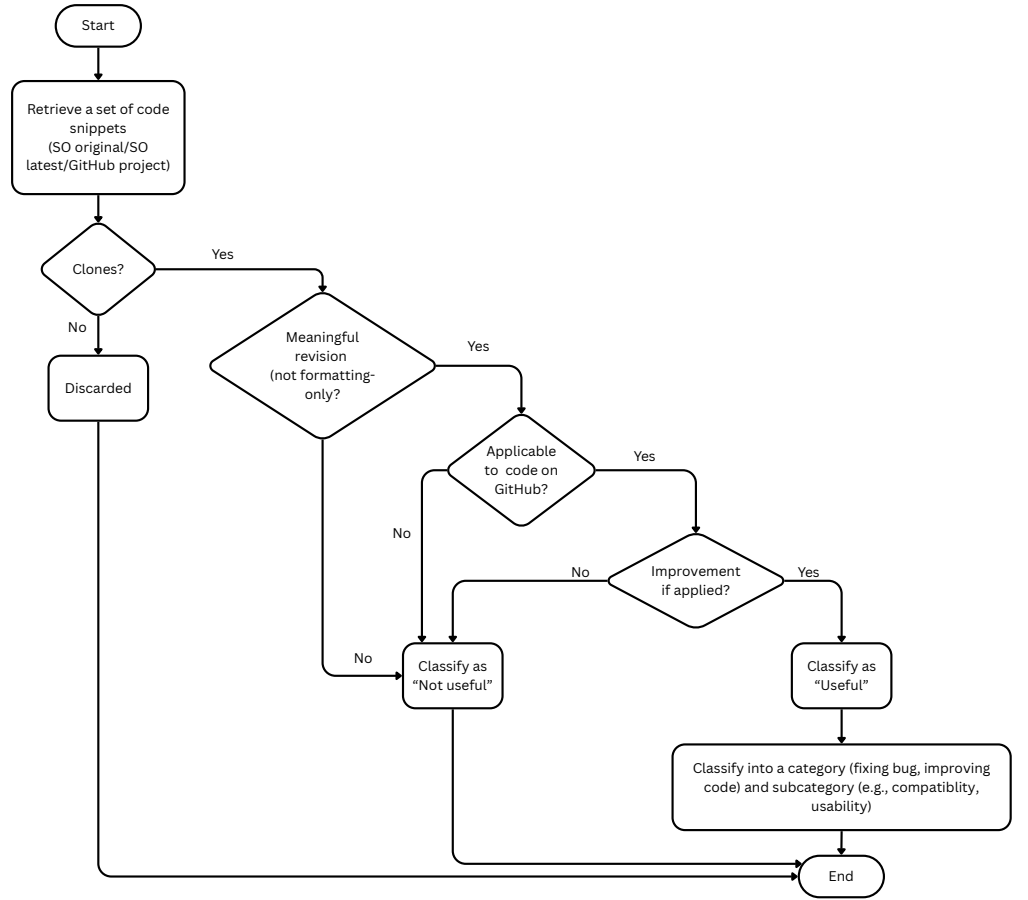


Fig 4. Manual code book for classifying potential clones

the action the user performed in the SO answer edit, such as **fixing**, **updating**, and **improving**, and the target of the edit, such as **formatting**, **code**, and **bug**. We adopted this classification and mapped the action performed to the edit target to derive the final category. Nonetheless, this categorisation is based solely on edits made to SO answers, including both text and code. Thus, many of the categories are not applicable to our study (e.g., fixing typos). Moreover, it does not consider the application of the code on software projects. As a result, we employed open coding to create additional categories based on observations of the changes required to adopt the latest SO answer into the software projects.

We followed a structured manual code book to classify each snippet pair (as shown in Figure 4). First, we retrieved a set of code snippets containing the original version on SO, the latest version on SO suggested by Siamese+, and the associated GitHub project’s code. We then verified the similarity between the original SO code and the latest SO code snippet (i.e., whether they were clones). If they are similar, we examined whether the latest SO revision introduced a meaningful change (not just formatting changes). Next, we checked whether it was applicable to the GitHub code. A pair was considered useful only if the revised snippet could be applied and resulted in a tangible improvement (e.g., bug fix or code quality enhancement). Otherwise, it was labeled as not useful. Lastly, we classified them into a category following Baltes et al. [38] and a subcategory.

The manual classification was performed independently by two authors (first and

Table 2. Categories of Stack Overflow Post Edits by [38]

Type	Category
Actions performed	adding, updating, deleting, fixing, improving, clarifying, simplifying, explaining, editing, copy-editing, active reading, refactoring
Target of the edit	formatting, typo, grammar, spelling, code, bug, link, image, example, syntax, solution, tag
Meta-level	sarcasm

fifth authors), who labelled each recommendation as applicable/non-applicable to their respective GitHub project and classified them into categories. After completing their independent classification, the authors compared their labelling, and the inter-rater reliability was measured using Cohen’s kappa coefficient. For recommendations with disagreements in classification, a third researcher provided a final classification. After the manual classification, we had a list of categories into which all potential clones had been categorised. Finally, all SO’s latest code answers that perform a relevant code update, such as **Fixing Bug** and **Improving code**, were considered applicable to open-source projects. The manual validation supports this qualitative analysis. The results obtained in this phase answer research questions RQ1 and RQ2.

3.3.3 Phase 3: Submitting Stack Overflow Latest Code Answers as Pull-Requests

To assess the usefulness of the SO’s latest code answers identified in Phase 2 to open-source software developers, we created and submitted pull requests to their associated GitHub projects. In the pull request, we made our best effort to implement minimal changes that incorporate the improvements based on SO’s latest code answers. The description for the pull request contains the explanation of this study and the ethics approval information. The information about the change was either adapted from the SO edit comment or created from scratch if no edit comment was provided. An example of such a pull request is shown in Figure 5.

Then, we monitored each pull request’s status and minimised interference with the review process whenever possible. This is necessary to mitigate any bias we may introduce into the review process.

4 Results

In this section, we discuss the findings from performing the [three phases](#) of our experimental design and provide the answers to the [two](#) research questions.

4.1 Study Setup

Before answering the research questions, we report two preparatory outcomes that configure our study pipeline: the optimised clone-search configuration obtained in Phase 1, and the filtered and grouped GitHub project dataset prepared in Phase 2.

4.1.1 The Optimised Siamese Configurations

Given our defined grid, the Grid Search algorithm evaluated a total of 3,073 configurations. This process was completed in 54 hours, with each configuration taking an average of 63

Update AddSubtractDaysSkippingWeekendsUtils.java #18370

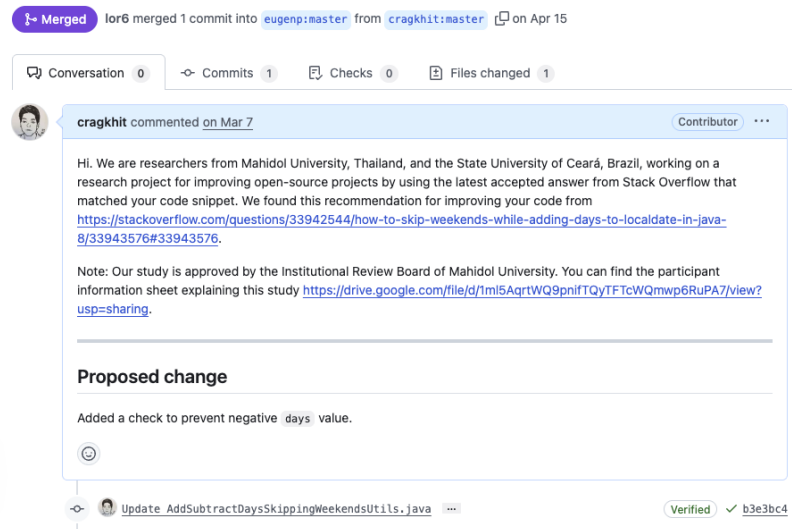


Fig 5. Example of a pull request submitted

Table 3. Optimised Configurations of Siamese

Parameters	Values
Clone size	6 lines
Size of n-gram	1 (for representation r0) 4 (for representation r1, r2, r3)
QR thresholds	9, 6, 5, 9 (r0, r1, r2, r3)
Similarity thresholds	50%,60%,70%,80% (r0, r1, r2, r3)

seconds. Tuning Siamese’s parameters using the SO data (72,365 Java code snippets) and the Qualitas dataset of 111 Java projects resulted in the optimized parameters shown in Table 3, which offer the highest MRR score of 0.782.

Examination of the results for the configuration depicted in Table 3 reveals several interesting facts. First, Siamese never returned an empty set as results, i.e., for all queries, a set of candidate snippets was always returned. Moreover, the correct clone was always present in the returned set of code snippets. Although the correct snippet was not always displayed in the first position, this occurred in only about 20% of the queries in our ground truth. An MRR value of 0.782 indicates that, in about 80% of the cases, the correct clone was ranked 1st. This optimised parameter helps reduce the number of false clones detected by Siamese and the time required for the manual validation step. We utilise this tuned configuration in Siamese+ to identify similar SO answers and open-source Java projects on GitHub.

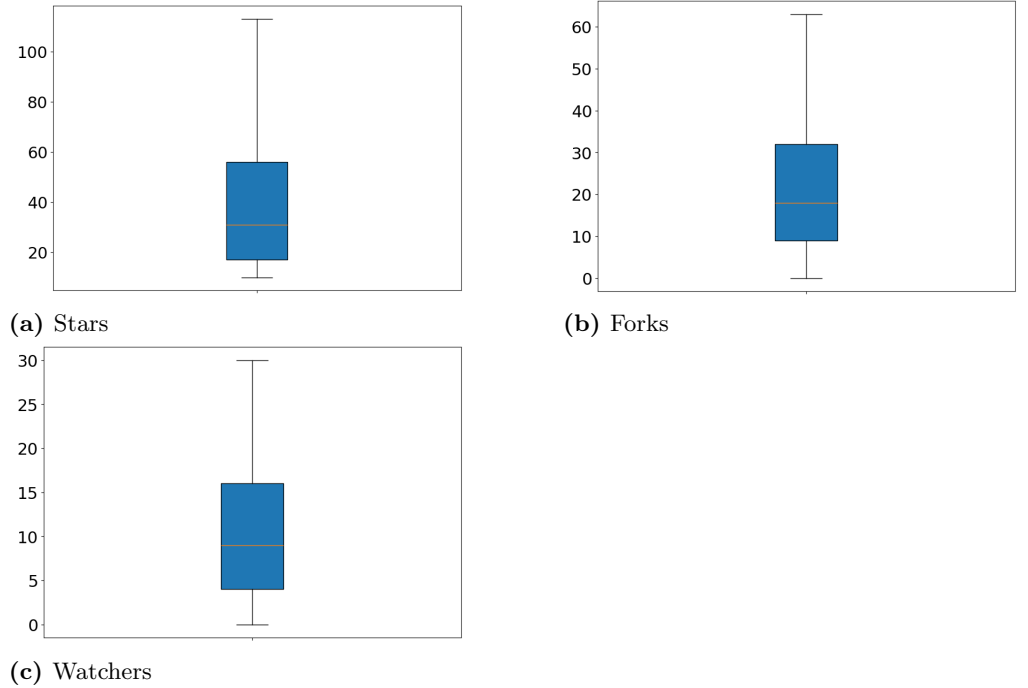
4.1.2 Filtered and Grouped GitHub Projects

After querying GHS (GitHub Search) [50] using the following filters: **Language: Java; Exclude Forks; Has Open Issues; Has Open Pull Requests**, we obtained 20,976 GitHub projects. Then, we analysed the projects based on the three popularity metrics: **Number of Stars, Number of Watchers, and Number of Forks**.

The statistics of the collected GitHub projects are shown in Table 4 and displayed as boxplots in Figure 6. For stars, the values range from 10, the minimum value of stars of projects in the GHS system, to 145,397. The first, second, third, and fourth quartiles

Table 4. Summary statistics for stargazers, forks, and watchers.

	Stars	Forks	Watchers
Mean	558.80	185.72	38.93
Std.	2,609.89	957.41	126.28
Min	10.00	0.00	0.00
25%	25.00	14.00	6.00
50% (Median)	70.00	34.00	13.00
75%	272.00	106.00	31.00
Max	145,397.00	50,497.00	5,442.00

**Fig 6.** Distribution of the metrics of collected GitHub projects

are between $[10, 25]$, $(25, 70]$, $(70, 272]$, and $(272, 145397]$ respectively. For forks, the values range from 0 to 50,497. The first, second, third, and fourth quartiles are between $[0, 14]$, $(14, 34]$, $(34, 106]$, and $(106, 50497]$ respectively. For watchers, the values range from 0 to 5,442. The first, second, third, and fourth quartiles are between $[0, 6]$, $(6, 13]$, $(13, 31]$, and $(31, 5442]$ respectively.

We filtered only the projects that passed the criteria articulated in Section 3.3.2, i.e., (1) the three metric values are all in the first quartile (low-popularity), (2) the three metric values are all in the fourth quartile (high-popularity), and (3) the three metrics values are within the second and the third quartiles (medium-popularity). This resulted in 10,673 GitHub projects containing 4,571,660 Java files. The number of projects in each group is shown in Table 5. The number of projects in the medium-popularity group is the highest (5,073), followed by the high-popularity (3,497) and low-popularity (2,103) groups. Moreover, the code sizes of the projects range from the low-popularity group (average code lines of 18k) to the medium-popularity group (46k) and the high-popularity group (113k) accordingly. Figure 7 shows the distribution of the project sizes in terms of the number of code lines in the three groups. As previously discussed, the number of code lines is largest in the high-popularity projects, indicating that they have larger

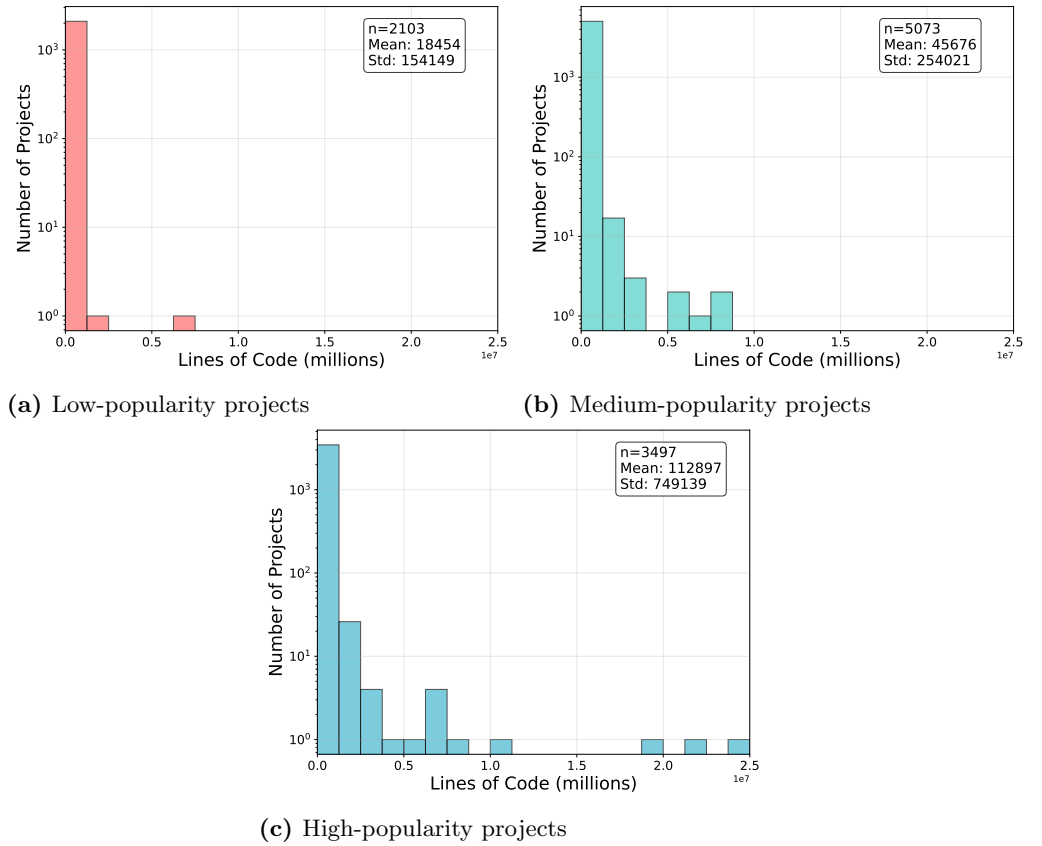


Fig 7. Distribution of the code lines in each project group

Table 5. Collected GitHub Projects Categorised by Groups

Group	Amount	Avg. Lines	Std. Dev.
Low-popularity	2,103	18,453.85	154,149.35
Medium-popularity	5,073	45,675.51	254,021.39
High-popularity	3,497	112,896.53	749,139.46

code bases than the low- and medium-popularity groups. These GitHub projects were used to search for the latest code answers based on the SO data.

4.2 RQ1: To what extent are accepted Java answers on Stack Overflow edited?

We identified 2,038,207 accepted Java answers on SO, sourced from SOTorrent, which contains 2,564,793 Java code snippets. This resulted in an average of 1.26 code snippets per Java accepted answer, with a standard deviation of 1.18. From these accepted answers, we filtered only those that had at least one revision (i.e., had been edited at least once) and revisions to the code blocks. This resulted in 140,840 answers (6.91%) that had more than one revision. The average number of revisions per answer was 2.82, with a median of 2.0. The total number of code snippets across all revisions of accepted answers was 283,838. The distribution of the revisions in our SO Java accepted answers is shown in Figure 8. We indexed these 283,838 answer revisions in Siamese+. The answer with the most revisions among our collected SO answers had 53 revisions.

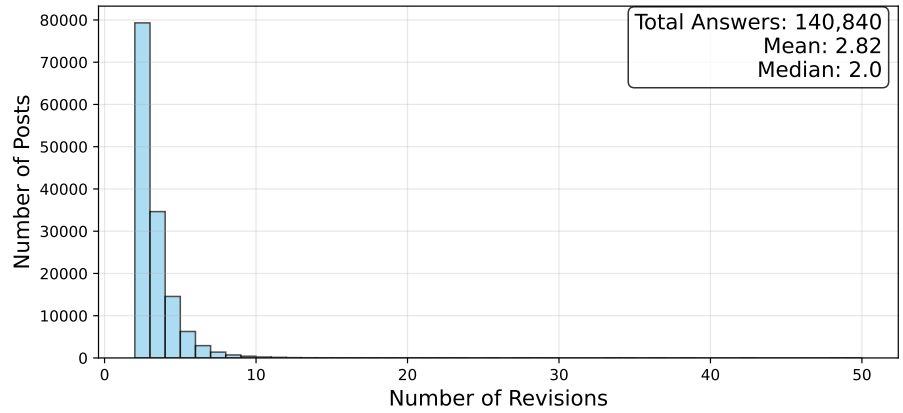


Fig 8. The distribution of revisions in our filtered Stack Overflow answers

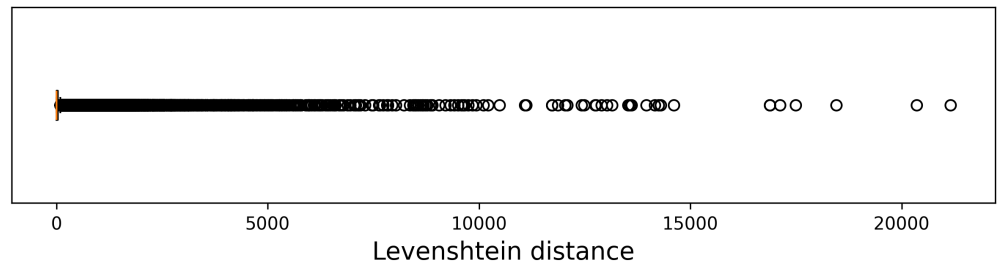


Fig 9. Distribution of the Levenshtein distance between the original and the latest SO code answers

The accepted answer with 53 revisions is the post ID 62805030¹⁵. It provides an explanation of multiple methods in the Apache Lang3 `StopWatch`, including `split()`, `stop()`, and `getSplitTime()`. It also provides example code snippets for using such methods. The first revision was created on July 8, 2020, at 23:14. The answerer then continued updating his/her answer until the last revision on January 24, 2021, at 1:10 (spanning 6 months and 17 days).

To gauge the size of code changes, we computed the Levenshtein distance, a metric that quantifies the similarity between two strings by counting the minimum number of single-character edits needed to transform one into the other. This comparison was made between the original code answer and their latest code answer to quantify the number of code changes. The distribution of the Levenshtein distance values is depicted in Figure 9 with the minimum distance of 0, maximum distance of 21,158 and the mean distance of 88.87 (standard deviation of 359.44). The accepted answer with the largest number of revisions is post ID 24407816¹⁶, where the code of `JideScrollPane` in the answer was heavily modified over two revisions. So, most of the SO code answer edits are small, no more than 100 characters, but there are also some large edits that modify the majority of the code answer.

¹⁵<https://stackoverflow.com/posts/62805030/revisions>

¹⁶<https://stackoverflow.com/posts/24407816/revisions>

Answer to RQ1: 6.91% of Stack Overflow Java accepted answers have more than one revision, with the average number of revisions per answer being 2.82. The average code edit size between the original and the latest SO revisions is 88.87 characters.

4.3 RQ2: What types of code improvements do Stack Overflow answer edits suggest for code found in open-source projects, and how are they distributed across projects of different popularity?

Siamese+ was used with the optimised configurations to generate recommendations for the 283,838 indexed SO answer revisions, using the 10,673 GitHub projects collected. The tool identified 793 pairs of code snippets in GitHub projects that included the latest code answers. Due to the large number of SO and GitHub code snippets, Siamese+ execution took 16 days, 5 hours, and 21 minutes.

4.3.1 Classification of Potential Clones

The first and the fifth authors, who each had at least four years of Java programming experience, performed a manual investigation on the 793 pairs independently to (1) validate whether the recommendations are applicable to the open-source project or not (i.e., they actually improve the code in GitHub projects), and (2) classify them into one of the categories defined by Baltes et al. [38]. Code answers that performed trivial changes, such as adding comments, were classified as non-applicable. After the manual investigation, the two validators compared their results. There were 298 latest SO code answers that the two authors agreed to be applicable, 408 SO answers that the two validators agreed to be not applicable, and 87 SO answers with disagreements. This yielded a Cohen's Kappa score of 0.8333, indicating almost perfect agreement. Then, the two validators and the second author discussed the applicability and classification of the 87 pairs for which there was disagreement until a consensus was reached. The final results contain 391 applicable recommendations and 402 non-applicable recommendations.

The classification results of the applicable latest SO code answers are shown in Table 6. The 391 useful recommendations are mapped to either *Fixing Bug* or *Improving Code*. Our findings are similar to the study by Jallow et al. [36], which classified Stack Overflow answer edits into security-relevant, bug fixes, and improvements. Since the Improving code category is coarse-grained, the two validators used open-coding to create an additional twelve subcategories of code improvement, including (1) application/framework specific, (2) code clarity and maintainability, (3) compatibility, (4) data and file handling, (5) data structures and algorithms, (6) error handling and robustness, (7) functional enhancement, (8) miscellaneous/others, (9) resource and process management, (10) security and cryptography, (11) usability, and (12) UI and interaction. The majority of the Improving Code recommendations fall into the *code clarity and maintainability* (57), followed by *application/framework-specific* (56), and *data and file handling* (52).

4.3.2 Distribution of the Applicable SO's Latest Code Answers

As shown in Table 5, we analysed the distribution of the SO's latest code answers across the groups of the collected GitHub projects (low-popularity, medium-popularity, and high-popularity). We counted the occurrences of SO's latest code answers in the two categories (i.e., Fixing Bugs and Improving Code) and divided them across the three GitHub groups. The results are depicted in Figure 10. Examining the Fixing Bug category, we noted that the number of SO's latest code answers increased from

Table 6. Results of manual type classification of the 391 applicable latest SO code answers

Baltes et al. [38] category	Subcategory	Amount
Fixing Bug		57
Improving Code		334
	Application/Framework-Specific	56
	Code Clarity and Maintainability	57
	Compatibility	7
	Data and File Handling	52
	Data Structures and Algorithms	14
	Error Handling and Robustness	45
	Functional Enhancement	21
	Miscellaneous/Others	21
	Resource and Process Management	14
	Security and Cryptography	2
	Usability	33
	UI and Interaction	12

the low-popularity group (5) to the medium-popularity group (15), and then to the high-popularity group (37). The same observation was found for the Improving Code category. As shown in Table 7, the maximum number of SO’s latest code answers in the Fixing Bug category is around 1 to 12 (average values between 0.0024–0.0106). For the Improving Code category, the maximum number of SO’s latest code answers is between 9 and 10 (average values between 0.0185 and 0.0495).

We performed a statistical test to assess differences in the number of SOs’ latest code answers in the Fixing Bug category across the three project groups. The null hypothesis (H_0) is *There is no statistically significant difference in the number of Fixing Bug recommendations across the three groups*. The Shapiro-Wilk normality test [64] suggested a non-normal distribution of the data across the three groups. Thus, we chose a non-parametric Kruskal-Wallis test [65]. The test reported a p-value of 0.038673 ($\alpha = 0.05$), a Kruskal-Wallis statistic (H) of 6.505224, and degrees of freedom of 2. Thus, we rejected the null hypothesis and concluded that there is a statistically significant difference between the Fixing Bug recommendations across the project quality groups.

We repeated the same test for the SO’s latest code answers in the Improving Code category. The null hypothesis (H_0) is defined similarly as *There is no statistically significant difference in the number of Improving Code recommendations across the three groups*. Since the data are not normally distributed, we again chose the nonparametric Kruskal-Wallis test. The test reported a p-value of 0.000016 ($\alpha = 0.05$), a Kruskal-Wallis statistic (H) of 22.100237, and degrees of freedom of 2. We rejected the null hypothesis, concluding that there is also a statistically significant difference in the Improving Code recommendations across the three project quality groups.

The results indicate that the latest answers on SO, particularly those related to bug fixes and code improvements, are more applicable to high-popularity open-source projects than to medium- and low-popularity projects.

Since we identified 12 subcategories within the Improving Code category, we also analysed their distributions across the three GitHub project groups. The result is shown in Table 8, where the largest percentage value in each group is highlighted. For the low-popularity group, the largest SO latest answers fall into the application/framework specific subcategory (28.21%). For the medium-popularity group, the largest portion (21.31%) of the SO’s latest answers is in the code clarity and maintainability subcategory.

Table 7. Number of SO Latest Code Answers Grouped by GitHub Project Groups (Max and Avg. are the maximum and average number per project, respectively)

Group	Fixing Bug			Improving Code		
	Total	Max	Avg.	Total	Max	Avg.
Low-popularity	5	1	0.0024	39	10	0.0185
Medium-popularity	15	1	0.0030	122	9	0.0240
High-popularity	37	12	0.0106	173	9	0.0495

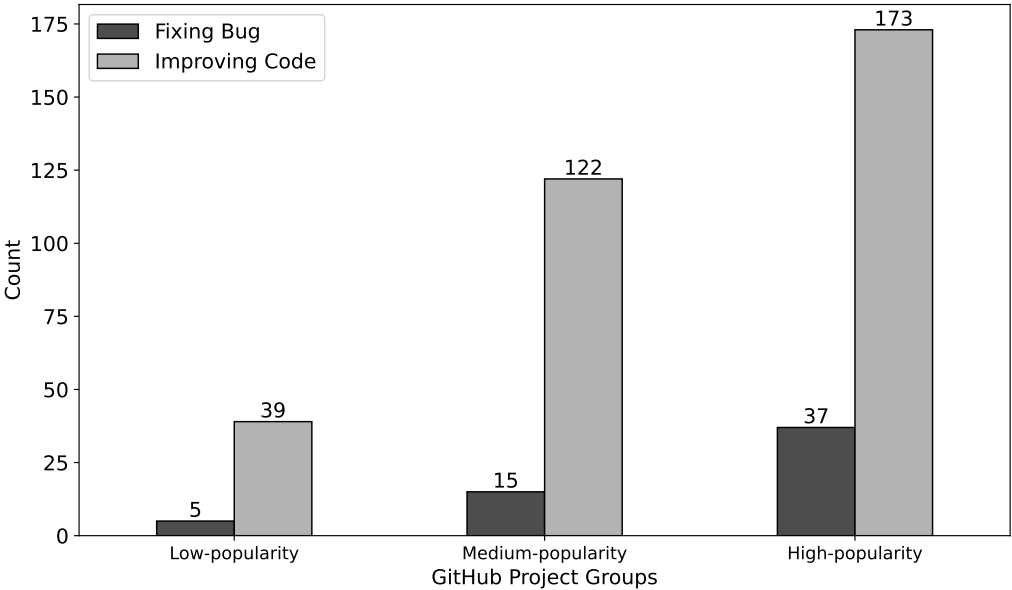


Fig 10. Code recommendations grouped by GitHub projects

Lastly, for the high-popularity group, the largest portion (19.08%) is for improving the code’s error handling and robustness.

Answer to RQ2: 49.30% (391 out of the 793) of SO’s latest code answers are considered applicable to the GitHub projects. The number of recommendations for both the Fixing Bugs and Improving Code categories is the highest in high-popularity GitHub projects.

4.4 Practical Application of the Approach (Phase 3)

To assess the usefulness of the SO’s latest code answers, we created pull requests to their respective GitHub projects, containing the latest code answers with modifications based on the 57 items in the Fixing Bug category. We refer to these SO’s latest code answers, submitted as pull requests, as “code recommendations.” We focused on the Fixing Bug recommendations because they offer clear benefits in improving the code and reducing the burden on OSS project maintainers. However, we had to ignore ten Fixing Bug recommendations because the GitHub projects or files no longer existed. Moreover, some of the pull requests contain multiple code recommendations for the same project, so we grouped them into a single pull request. Thus, finally, we opened 24 pull requests. The details of the pull requests are shown in Table 9. Looking at the GitHub project associated with the pull requests, there are 12 high-popularity-group projects (marked

Table 8. Distribution of the Improving Code Items Across GitHub Project Groups

Category	Low-popularity ($N = 39$)	Medium-popularity ($N = 122$)	High-popularity ($N = 173$)
Error Handling & Robustness	3 (7.69%)	9 (7.38%)	33 (19.08%)
Code Clarity & Maintainability	6 (15.38%)	26 (21.31%)	25 (14.45%)
Application/Framework-Specific	11 (28.21%)	21 (17.21%)	24 (13.87%)
Data & File Handling	7 (17.95%)	25 (20.49%)	20 (11.56%)
Usability	3 (7.69%)	15 (12.30%)	15 (8.67%)
Functional Enhancement	1 (2.56%)	6 (4.92%)	14 (8.09%)
Resource & Process Management	1 (2.56%)	1 (0.82%)	12 (6.94%)
Data Structures & Algorithms	0 (0.00%)	4 (3.28%)	10 (5.78%)
UI & Interaction	2 (5.13%)	1 (0.82%)	9 (5.20%)
Miscellaneous / Others	5 (12.82%)	8 (6.56%)	8 (4.62%)
Compatibility	0 (0.00%)	4 (3.28%)	3 (1.73%)
Security & Cryptography	0 (0.00%)	2 (1.64%)	0 (0.00%)

with H in the table), eight medium-popularity-group projects (marked with M in the table), and four low-popularity-group projects (marked with L in the table).

For each pull request, we studied the differences between the original code and the SO’s latest code answer. We then cloned the project, created a new branch for the proposed update, and applied the changes based on the SO’s latest code answer, making adjustments to align with the existing code. Lastly, we created a pull request that included the study’s purpose (along with the ethics approval) and a brief explanation of the changes.

Since the creation of the first pull request (27 January 2025) to the time of writing of this paper, there have been four pull requests containing 11 code recommendations that were accepted by the GitHub project maintainers. There were 3 pull requests directly merged by the developers, and 1 pull request with 8 code recommendations, for which the proposed changes triggered the updates.

For the 20 unmerged pull requests (see Table 10), 17 of them are still pending for review, 1 pull request was closed with a suggestion to follow the project’s procedure (which we followed but did not get any further response), and 1 pull request was closed without any comment from the maintainer. Lastly, 1 pull request is currently under review (i.e., we have received a response from the reviewer, but it has not yet been merged).

4.4.1 Examples of the merged pull requests

We discuss some of the merged pull requests based on the code recommendations from our study. The pull request to the bcgit project, a high-popularity-group project with 1,868 stars, 1026 forks, and 126 watchers, contains 8 recommendations¹⁷. All the changes are made within the `isEqual` method by introducing an array object comparison and a null check before the comparison to avoid a `NullPointerException`, as shown in Figure 11. After we introduced the first recommendation to the file `core/src/test/java/org/bouncycastle/crypto/test/PKCS12Test.java` in the pull request, the owner of the bcgit project agreed with the changes and mentioned that they would switch to use another method called `areEqual` instead (“*Thanks for the PR. I have instead removed this `isEqual` method and referenced `org.bouncycastle.util.Arrays#areEqual` instead.*”). We added seven more similar recommendations as comments in the pull requests, and all were accepted. The full discussion of this pull request is in the Appendix. Since this project has 1,026 forks, the old buggy version of the code snippets that could

¹⁷<https://github.com/bcgitt/bc-java/pull/2012>

Table 9. List of the 24 opened pull requests from this study, their associated GitHub projects, and their statuses. The H, M, and L indicate high-popularity, medium-popularity, and Low-popularity project groups. For the statuses, U, M, and C denote Under Review, Merged, and Closed (without merging) accordingly.

No	GitHub Project	stars/forks/watchers	Group	Pull Request	Status
1	actiontech/dbble	1005/314/75	H	actiontech/dbble/pull/3898	U
2	aionnetwork/aion	337/113/83	H	aionnetwork/aion/pull/1177	U
3	leapframework/framework	43/19/10	M	leapframework/framework/pull/69	U
4	allwefantasy/serviceframework	549/271/164	H	allwefantasy/ServiceFramework/pull/96	U
5	threerings/getdown	481/122/47	H	threerings/getdown/pull/281	U
6	bcgit/bc-java	1868/1026/126	H	bcgit/bc-java/pull/2012	M
7	chrislacy/tweetlanes	779/296/77	H	chrislacy/TweetLanes/pull/443	U
8	clouway/oauth2-server	44/21/24	M	clouway/oauth2-server/pull/100	M
9	cryptoguardoss/cryptoguard	88/27/7	M	CryptoGuardOSS/cryptoguard/pull/35	U
10	cyriux/mpcmaid	24/6/4	L	cyriux/mpcmaid/pull/16	U
11	eugenp/tutorials	31934/50497/1543	H	eugenp/tutorials/pull/18370	M
12	giloutho/logfly5	18/6/5	L	giloutho/Logfly5/pull/18	U
13	linkedin/rest.li	2251/517/190	H	linkedin/rest.li/pull/1078	U
14	meituan-dianping/zebra	2536/696/167	H	Meituan-Dianping/Zebra/pull/114	U
15	mycatapache/mycat2	1298/438/91	H	MyCATApache/Mycat2/pull/824	U
16	mycatapache/mycat-server	9533/3889/676	H	MyCATApache/Mycat-Server/pull/2958	U
17	PGMDev/PGM	152/86/17	M	PGMDev/PGM/pull/1486	C
18	SCADA-LTS/Scada-LTS	481/225/67	H	SCADA-LTS/Scada-LTS/pull/3094	C
19	sergueik/selenium.java	21/11/5	L	sergueik/selenium.java/pull/431	U
20	teamapps-org/teamapps	18/9/5	L	teamapps-org/teamapps/pull/130	U
21	ukwa/webarchive-discovery	100/24/23	M	ukwa/webarchive-discovery/pull/321	M
22	vipshop/pallas	240/83/24	M	vipshop/pallas/pull/275	U
23	xnnyygn/xraft	184/90/7	M	xnnyygn/xraft/pull/46	U
24	zitmen/thunderstorm	54/30/16	M	zitmen/thunderstorm/pull/76	U

Table 10. Statuses of the unmerged pull requests

Status	Count
Pending for review	17
Closed	2
Under review	1
Total	20

cause a `NullPointerException` was adopted in these forked projects as well, and our recommendations fixed these issues.

Another example of a merged PR is the `oauth2-server` project, which is classified as a medium-popularity-group project with 44 stars, 21 forks, and 24 watchers¹⁸. We proposed a change in the `clouway/oauth2/app/AppServer.java` file as shown in Figure 12. The recommendation adds a null check before stopping the server. The PR was merged by the project maintainer in just 2 minutes.

These pull request results demonstrate that the Fixing Bug code recommendations are perceived as useful by the open-source software developers and are accepted for merging into their software projects. A few additional examples of the pull requests can be found in the Appendix.

4.4.2 Comparison to Existing Studies

To put our acceptance rate into perspective, we compare it with related studies that also validate automated or semi-automated code improvements through real pull requests. Our 36 Fixing Bug recommendations were grouped into 24 pull requests, of which four pull requests (containing 11 recommendations) were accepted, corresponding to an acceptance rate of 30.6% (11/36) at the level of individual recommendations. The most

¹⁸<https://github.com/clouway/oauth2-server/pull/100>

```

1 @@ -26,6 +26,13 @@ private boolean isEqual(
2     byte[] a,
3     byte[] b)
4     {
5 +         if(a == b){
6 +             return true;
7 +         }
8 +         if(null == a || null == b){
9 +             return false;
10 +        }
11 +
12     if (a.length != b.length)
13     {
14         return false;
15     }
16     for (int i = 0; i != a.length; i++)
17     {
18         if (a[i] != b[i])
19         {
20             return false;
21         }
22     }
23     return true;
24 }
25

```

Fig 11. Example of a Fixing Bug recommendation in the pull request of the bcgit project

```

1 @@ -43,12 +43,14 @@ protected Injector getInjector() {
2     Runtime.getRuntime().addShutdownHook(new Thread() {
3         @Override
4         public void run() {
5 -         try {
6 -             server.stop();
7 -         } catch (Exception e) {
8 -             e.printStackTrace();
9 +             if (server != null) {
10 +                 try {
11 +                     server.stop();
12 +                 } catch (Exception e) {
13 +                     e.printStackTrace();
14 +                 }
15             }
16         }
17     });
18 }
19 }
20 }
21

```

Fig 12. Example of a bug fix recommendation in the pull request of the oauth2-server project.

directly comparable study is that of Tang and Nadi [26], who similarly mined Stack Overflow answer edits (from comment-edit pairs) and validated them by submitting 15 pull requests, of which 10 were accepted. Although their acceptance ratio is higher, their submissions were a small, hand-selected set targeting a few well-known repositories, whereas our pull requests were submitted as unsolicited, external contributions to projects spanning different popularity levels.

Beyond Stack Overflow-based approaches, techniques that deliver automated or AI-assisted changes report substantially higher acceptance rates, but under conditions that strongly favour acceptance. For instance, the B-Assist automated program repair tool deployed at Bloomberg reached a 74.56% acceptance rate by presenting patch suggestions in context within active pull requests to the engineers most familiar with the affected code [66], and a recent study of agentic (LLM-generated) pull requests reports that 83.8% were eventually merged [67].

These settings differ fundamentally from ours, in which improvements are proposed by an external party with no prior relationship to the project. This distinction matters because pull requests from external contributors are known to be rejected far more often than those from project insiders. Pinto et al. [68] found that volunteers face roughly 26 times more rejection than employees contributing to the same projects. Viewed against these existing studies, and given that our submissions were unsolicited, externally submitted, and derived from Stack Overflow edits, our 30.6% acceptance rate represents an encouraging result rather than an underperformance.

5 Discussion

5.1 Implications for software engineering researchers

The findings from this study corroborate those of related studies on answer edits on Stack Overflow [26, 36, 39]. We similarly found that outdated answers on SO are still being used in open-source software projects, even though newer versions address the issues of concern. [The results from the practical application of our approach \(Section 4.4\) show](#) that the identified latest SO code answers are applicable to real-world open-source software projects and were accepted by the project maintainers. Future research can focus on developing an automated tool, based on the design of our Siamese+ tool, that recommends code improvements for open-source software projects using crowd-sourced SO answers. Moreover, our manual classifications show that not all of the latest SO code answers are applicable to GitHub software projects. Thus, future studies can develop machine learning models or leverage LLMs to classify whether the latest SO code answers are applicable to software projects, thereby reducing the number of false-positive recommendations proposed to developers.

5.2 Implications for open-source software maintainers

Our study and related studies reveal that code answers on SO change over time [25–28, 36, 39]. Some of the original answers may contain bugs or security weaknesses. Thus, it is important that open-source project maintainers keep track of code reused from SO, potentially by adding a comment with attribution and a link to the SO post or answer the code is copied from. By adding comments, they can easily track whether reused code from SO remains up to date. Moreover, they can adopt the Siamese+ tool used in this study to analyse their codebase against the SO answer revision data.

5.3 Code recommendations from SO in the era of Generative AI

With the rise of Generative AI (GenAI) tools, especially models specifically designed for code like GitHub’s Copilot or Claude Code, generating code and fixing bugs have become trivial tasks. However, recent studies reveal that code snippets generated by these GenAI tools are not always high-quality [69–75]. They may contain subtle bugs, maintenance issues, suboptimal code, or security vulnerabilities. Thus, it is important that the developers verify their correctness before integrating them into the code bases. We believe the findings from our study can complement this ongoing trend toward AI-assisted programming. By leveraging crowd-sourced knowledge from SO, developers can manually review the code generated by GenAI tools and verify it against the latest answers on the platform.

Notably, a recent study by Helic et al. [22] reports that the questions and answers on SO have evolved with the emergence of GenAI tools. The recent questions and answers on SO address more complex and challenging programming problems than in the past because they are the problems that cannot be handled by GenAI. Thus, we foresee that these code improvement recommendations based on SO will remain useful in the near future, particularly for complex code that cannot be fixed by GenAI.

5.4 Limitations of leveraging Stack Overflow answer revisions

While our results show that SO answer revisions can yield useful code improvements, leveraging them for software maintenance has inherent limitations along three dimensions. First, regarding *code security*, SO answer revisions are crowd-sourced and are not systematically vetted for security. An edit that fixes a functional bug does not necessarily address security weaknesses, and prior work has shown that widely reused SO snippets can contain vulnerabilities that persist across revisions [36, 76]. Adopting the latest SO revision is therefore not a guarantee of a secure fix, and in the worst case an edit could introduce a new weakness. For this reason, our approach treats a revision as a recommendation to be reviewed by a human maintainer rather than as a patch to be applied automatically. Second, regarding *project-specific requirements*, SO answers are generic, context-free solutions to isolated programming problems, whereas real projects impose constraints such as specific API and library versions, coding conventions, performance and concurrency assumptions, and architectural decisions. An edit that is a genuine improvement in isolation may not fit a particular project, may conflict with its style, or may require non-trivial adaptation before it can be merged; this partly explains why not all applicable recommendations were accepted. Third, regarding *evolving programming practices*, both SO content and the surrounding language and library ecosystems evolve over time, so a “latest” SO revision reflects the best practice only at the time of that edit and may itself become outdated as new language features are introduced, APIs are deprecated, and conventions change. This limitation is compounded by the temporal coverage of our SOTorrent dataset (Section 6). Taken together, these limitations define the setting in which our approach is most appropriate: a semi-automated recommender that surfaces candidate improvements for human review, rather than an autonomous maintenance tool.

6 Threats to the Validity

There are potential threats to the validity of our study, which we categorise into *construct*, *internal*, *conclusion*, and *external* threats as follows.

Construct validity: A key assumption underlying our study is that a matched GitHub snippet is an adoption of an earlier Stack Overflow answer that has since been

improved on SO. Our clone-based approach, however, establishes textual similarity between the two, not the direction or the origin of the reuse. A matched pair could instead reflect independent implementations of a common programming task, code copied from GitHub to SO, or reuse from a shared third-party source. We mitigated this threat by manually inspecting each candidate pair and retaining only those in which the latest SO revision constitutes a plausible, applicable improvement over the code in the GitHub project. Nevertheless, we cannot confirm the reuse direction for every pair, and our findings should be interpreted as evidence of applicable improvements rather than of confirmed SO-to-GitHub reuse.

Moreover, we show the usefulness of a recommendation through the acceptance of its corresponding pull request. Pull-request acceptance is influenced by factors beyond code quality, such as the maintainers' availability and responsiveness, the project's activity level, its contribution guidelines, and the perceived reputation of the contributor. Consequently, a rejected or unanswered pull request does not necessarily indicate that a recommendation is not useful, and vice versa. Lastly, we use the Levenshtein distance between the original and latest answers as a proxy for the magnitude of a code change. As a character-level metric, it is sensitive to superficial edits such as formatting, whitespace, and identifier renaming, and may therefore not fully reflect the semantic significance of a change.

Internal validity: A potential threat to internal validity arises from the adoption of the Siamese code clone search tool to identify similar code snippets in SO accepted answers, which may yield false positives and false negatives. We mitigated this threat in Phase 1 of our study by searching for optimised configurations of the tool to reduce the number of false results. Nonetheless, the optimised Siamese configuration is based on the Qualitas corpus, which dates back to 2013. Thus, it is possible that the optimised configurations may miss some of the Java language features and constructs released after 2013. Additionally, GitHub's popularity metrics (i.e., stars, watchers, forks) may not accurately reflect a project's code quality. Nevertheless, we argue that a categorisation criterion combining all three metrics to create three project groups may be a more trustworthy proxy than relying on any single metric alone. Another threat is that the manual validation step may contain errors due to human mistakes and biases. We mitigated this threat by employing two validators and cross-checking the results. The second author was involved in resolving disagreements. Moreover, the SO data used in this study is based on the SOTorrent dataset version 2020-12-31, which may differ from the latest Stack Overflow data. Finally, because we both created the pull requests and interpreted their outcomes, the framing of our submissions, each of which disclosed the study's purpose and ethics approval, may have influenced the maintainers' willingness to review or merge them. We attempted to minimise this effect by keeping the pull-request descriptions factual and by not intervening in the review process, but a potential bias may remain.

Conclusion validity: Our analysis of the distribution of recommendations across the low-, medium-, and high-popularity project groups is descriptive, reporting counts and proportions without formal significance testing. Accordingly, the observed differences between the groups should be interpreted as trends rather than as statistically confirmed effects.

External validity: The conclusions drawn from this study are based on the selection of GitHub projects according to our predefined methodology, detailed in Phase 1. Due to possible selection bias, the conclusions may not generalise to all GitHub projects. Moreover, we analysed only code written in Java across GitHub and SO answers. Hence, the findings may not be applied to other programming languages. Additionally, the number of 24 pull requests based on the Fixing Bug recommendations is quite small. Thus, the findings from this pull request evaluation may not be representative of all

the recommendations found in this study. Finally, since this study focused on accepted answers, the findings may not be applicable to other types of SO answers (e.g., newest answers, highest-voted answers). In addition, the SOTorrent dataset captures Stack Overflow activity only up to 31 December 2020, predating the widespread adoption of generative AI coding assistants. Editing behaviour and reuse dynamics on Stack Overflow may have shifted since then, so our findings reflect a historical snapshot and may not fully generalise to the current state of the platform.

7 Conclusion

This paper has presented an empirical study of edits made to Stack Overflow Java accepted answers and used them to provide recommendations for open-source software projects. We analysed 2,038,207 Java accepted answers on Stack Overflow and found that 6.91% of them had been updated, i.e., contained more than one revision. The average number of revisions per accepted SO answer is 2.82, but it can reach 53. By employing our specialised code clone search tool, called Siamese+, which can search Java code bases for similar code snippets across multiple revisions of Stack Overflow accepted answers and recommends code changes based on the latest code answer version, we found 793 code snippets in 10,673 GitHub Java projects that could benefit from the latest code updates on Stack Overflow. Our manual validation indicates that the latest code answers on Stack Overflow primarily fall into two categories: bug fixes and code improvements, found mostly in high-popularity GitHub projects. We created pull requests to GitHub projects that matched our latest Stack Overflow code answers, and 11 of our code recommendations from four pull requests were accepted by the project maintainers, demonstrating their practical usefulness in real-world software projects.

The results from this study show that Stack Overflow is a useful source for creating Java code improvement recommendations based on its latest answer edits. Due to Stack Overflow’s community-driven nature, answers are edited and improved over time, and these edits can be used to improve suboptimal code in software projects, whether the code was created independently or copied from Stack Overflow. The approach we used to locate potential clones between Stack Overflow and Java projects, i.e., Siamese+, has paved the way for future studies on automated code recommendation tools. We foresee this automated tool being integrated into the code review process, where each new code update is checked against existing Stack Overflow answers and recommended updates are provided if needed.

8 Declarations

8.1 Funding

The research project was partially supported by Scholarships for Ph.D. student from Mahidol University and by a grant from the Office of the Permanent Secretary, Ministry of Higher Education, Science, Research and Innovation (RGNS64-143). The funders had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

8.2 Ethical approval

This project has been approved by the Mahidol University Central Institutional Review Board (MU-CIRB) with exemption. The protocol number is MU-CIRB 2024/116.2203.

8.3 Informed consent

Not applicable

8.4 Author Contributions

- In-on Wiratsin: Conceptualization, Data Curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing - Original Draft, Writing - Review & Editing
- Chaiyong Ragkhitwetsagul: Conceptualization, Data Curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing - Original Draft, Writing - Review & Editing
- Mathues Paixao: Conceptualization, Formal analysis, Investigation, Methodology, Supervision, Validation, Writing - Original Draft, Writing - Review & Editing
- Denis De Sousa: Conceptualization, Data Curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing - Original Draft, Writing - Review & Editing
- Pongpop Lapvikai: Investigation, Methodology, Validation
- Peter Haddawy: Conceptualization, Methodology, Writing - Review & Editing

8.5 Data Availability Statement

The authors declare that the data supporting the findings of this study are available within the paper and its supplementary information files. The code scripts used for the analysis are publicly available at <https://doi.org/10.5281/zenodo.17220745>.

8.6 Conflict of Interest

Not applicable

References

1. Shrestha N, Botta C, Barik T, Parnin C. Here We Go Again: Why is It Difficult for Developers to Learn Another Programming Language? In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20); 2020. p. 691–701.
2. Eyolfson J, Tan L, Lam P. Do time of day and developer experience affect commit bugginess. In: Proceedings of the 8th Working Conference on Mining Software Repositories (MSR '11); 2011. p. 153.
3. Kuuttila M, Mäntylä M, Farooq U, Claes M. Time pressure in software engineering: A systematic review. Information and Software Technology. 2020 May;121:106257.
4. Lavallee M, Robillard PN. Why Good Developers Write Bad Code: An Observational Case Study of the Impacts of Organizational Factors on Software Quality. In: Proceedings of the 37th International Conference on Software Engineering (ICSE '15). vol. 1; 2015. p. 677-87.

5. Freire S, Rios N, Gutierrez B, Torres D, Mendonça M, Izurieta C, et al. Surveying Software Practitioners on Technical Debt Payment Practices and Reasons for not Paying off Debt Items. In: Proceedings of the Evaluation and Assessment in Software Engineering (EASE '20); 2020. p. 210-9.
6. Graziotin D, Fagerholm F, Wang X, Abrahamsson P. Unhappy Developers: Bad for Themselves, Bad for Process, and Bad for Software Product. In: Proceedings of the 39th International Conference on Software Engineering Companion (ICSE-C '17); 2017. p. 362-4.
7. Sjoberg DIK, Yamashita A, Anda BCD, Mockus A, Dyba T. Quantifying the Effect of Code Smells on Maintenance Effort. *IEEE Transactions on Software Engineering*. 2013 aug;39(8):1144-56.
8. Tom E, Aurum A, Vidgen R. An exploration of technical debt. *Journal of Systems and Software*. 2013 Jun;86(6):1498-516.
9. Bavota G, Russo B. A large-scale empirical study on self-admitted technical debt. In: Proceedings of the 13th International Conference on Mining Software Repositories (MSR '16); 2016. p. 315-26.
10. Yang D, Martins P, Saini V, Lopes C. Stack Overflow in GitHub: Any Snippets There? In: Proceedings of the 14th International Conference on Mining Software Repositories (MSR '17); 2017. .
11. Ragkhitwetsagul C, Krinke J. Siamese: scalable and incremental code clone search via multiple code representations. *Empirical Software Engineering*. 2019 8;24:2236-84.
12. Zhang H, Wang S, hsun Peter Chen T, Zou Y, Hassan AE. An Empirical Study of Obsolete Answers on Stack Overflow. *IEEE Transactions on Software Engineering*. 2019:850-62.
13. Chen J, Zhao Y, Meng N. How Do Developers Reuse StackOverflow Answers in Their GitHub Projects? In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops. New York, NY, USA: Association for Computing Machinery; 2024. p. 146–155. doi:10.1145/3691621.3694945.
14. Huang Y, Xu F, Zhou H, Chen X, Zhou X, Wang T. Towards exploring the code reuse from stack overflow during software development. In: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. vol. 2022-March. ACM; 2022. p. 548-59.
15. Chen X, Xu F, Huang Y, Zhou X, Zheng Z. An empirical study of code reuse between GitHub and Stack Overflow during software development. *Journal of Systems and Software*. 2024;210:111964. doi:https://doi.org/10.1016/j.jss.2024.111964.
16. Abdalkareem R. Studying the Role of Reusing Crowdsourcing Knowledge in Software Development; 2025. Available from: <https://arxiv.org/abs/2512.07824>. arXiv:2512.07824.
17. Yu C, Noei S, Zhang H, Zou Y. An Empirical Study on the Characteristics of Reusable Code Clones. *ACM Trans Softw Eng Methodol*. 2026 Jan. Available from: <https://doi.org/10.1145/3793251>. doi:10.1145/3793251.
18. Burtch G, Lee D, Chen Z. The consequences of generative AI for online knowledge communities. *Scientific Reports*. 2024;14. doi:https://doi.org/10.1038/s41598-024-61221-0.

19. del Rio-Chanona RM, Laurentsyeve N, Wachs J. Large language models reduce public knowledge sharing on online Q&A platforms. *PNAS Nexus*. 2024 09;3(9):pgae400. doi:10.1093/pnasnexus/pgae400.
20. Xue J, Wang L, Zheng J, Li Y, Tan Y. Can ChatGPT Kill User-Generated Q&A Platforms? *Information Systems Research*. 2026. Articles in Advance. doi:10.1287/isre.2023.0561.
21. Shan G, Qiu L. Examining the Impact of Generative AI on Users' Voluntary Knowledge Contribution: Evidence from a Natural Experiment on Stack Overflow. *Information Systems Research*. 2025. doi:10.1287/isre.2023.0332.
22. Helic D, Santos T. Stack Overflow is not dead yet: Crowd answers still matter. *Journal of Systems and Software*. 2026;241:112988. doi:https://doi.org/10.1016/j.jss.2026.112988.
23. Acar Y, Backes M, Fahl S, Kim D, Mazurek ML, Stransky C. You Get Where You're Looking for: The Impact of Information Sources on Code Security. In: *IEEE Symposium on Security and Privacy (SP '16)*; 2016. p. 289-305.
24. Zhang T, Upadhyaya G, Reinhardt A, Rajan H, Kim M. Are code examples on an online Q&A forum reliable? In: *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*; 2018. p. 886-96.
25. Diamantopoulos T, Sifaki MI, Symeonidis A. Towards Mining Answer Edits to Extract Evolution Patterns in Stack Overflow. In: *Proceedings of the 16th International Conference on Mining Software Repositories (MSR '19)*. vol. 2019-May; 2019. p. 216-9.
26. Tang H, Nadi S. On using Stack Overflow comment-edit pairs to recommend code maintenance changes. *Empirical Software Engineering*. 2021 Jul;26(4):68.
27. Sheikhaei MS, Tian Y, Wang S. A study of update request comments in Stack Overflow answer posts. *Journal of Systems and Software*. 2023 4;198:111590.
28. Mondal S, Roy CK. Does Editing Improve Answer Quality on Stack Overflow? A Data-Driven Investigation. In: *Proceedings of the 41st International Conference on Software Maintenance and Evolution (ICSME'25)*; 2025. .
29. Zuo S, Yang Q, Gao T, Proggä ZT, Shen J. How Security Coding Knowledge Impact Software Quality: An Empirical Study of Stack Overflow Security Issues. In: *2025 25th International Conference on Software Quality, Reliability and Security (QRS)*; 2025. p. 727-38. doi:10.1109/QRS65678.2025.00076.
30. Jallow A, Bugiel S. Stack Overflow Meets Replication: Security Research Amid Evolving Code Snippets. In: *Proceedings of the 34th USENIX Security Symposium (USENIX Security'25)*. USENIX Association; 2025. Extended version: arXiv:2501.16948.
31. Keivanloo I, Rilling J, Zou Y. Spotting working code examples. In: *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*; 2014. p. 664-75.
32. Ponzanelli L, Bacchelli A, Lanza M. Seahawk: Stack Overflow in the IDE. In: *Proceedings of the 2013 International Conference on Software Engineering (ICSE '13)*; 2013. p. 1295-8.

33. Ponzanelli L, Bavota G, Di Penta M, Oliveto R, Lanza M. Mining StackOverflow to turn the IDE into a self-confident programming prompter. In: Proceedings of the 14th International Conference on Mining Software Repositories (MSR '14); 2014. p. 102-11.
34. Treude C, Robillard MP. Augmenting API documentation with insights from stack overflow. In: Proceedings of the 38th International Conference on Software Engineering (ICSE '16). ACM Press; 2016. p. 392-403.
35. Gao Z, Xia X, Lo D, Grundy J, Zhang X, Xing Z. I Know What You Are Searching for: Code Snippet Recommendation from Stack Overflow Posts. *ACM Transactions on Software Engineering and Methodology*. 2023 7;32:1-42. doi:10.1145/3550150.
36. Jallow A, Schilling M, Backes M, Bugiel S. Measuring the Effects of Stack Overflow Code Snippet Evolution on Open-Source Software Security. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP'24). IEEE; 2024. p. 1083-101. doi:10.1109/SP54263.2024.00022.
37. Wang S, Chen TH, Hassan AE. How Do Users Revise Answers on Technical Q&A Websites? A Case Study on Stack Overflow. *IEEE Transactions on Software Engineering*. 2020 9;46:1024-38. doi:10.1109/TSE.2018.2874470.
38. Baltes S, Wagner M. An annotated dataset of stack overflow post edits. In: Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion (GECCO '20). vol. 323. ACM; 2020. p. 1923-5.
39. Zhang H, Wang S, Li H, Chen TH, Hassan AE. A Study of C/C++ Code Weaknesses on Stack Overflow. *IEEE Transactions on Software Engineering*. 2022 7;48:2359-75. doi:10.1109/TSE.2021.3058985.
40. Rahman MM, Roy CK, Lo D. RACK: Automatic API Recommendation Using Crowdsourced Knowledge. In: Proceedings of the 23rd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER '16); 2016. p. 349-59.
41. Rubei R, Di Sipio C, Nguyen PT, Di Rocco J, Di Ruscio D. PostFinder: Mining Stack Overflow posts to support software developers. *Information and Software Technology*. 2020 Nov;127:106367.
42. Nyamawe AS, Liu H, Niu Z, Wang W, Niu N. Recommending Refactoring Solutions Based on Traceability and Code Metrics. *IEEE Access*. 2018;6:49460-75.
43. Jin Y, Bai Y, Zhu Y, Sun Y, Wang W. Code Recommendation for Open Source Software Developers. In: Proceedings of the ACM Web Conference 2023. ACM; 2023. p. 1324-33. doi:10.1145/3543507.3583503.
44. Luan S, Yang D, Barnaby C, Sen K, Chandra S. Aroma: Code Recommendation via Structural Code Search. *Proceedings of the ACM on Programming Languages*. 2019;3(OOPSLA):1—28.
45. Guo Q, Cao J, Xie X, Liu S, Li X, Chen B, et al. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ACM; 2024. p. 1-13.
46. Baltes S, Dumani L, Treude C, Diehl S. SOTorrent: Reconstructing and Analyzing the Evolution of Stack Overflow Posts. In: Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18); 2018. p. 319-30.

47. Manes SS, Baysal O. How Often and What StackOverflow Posts Do Developers Reference in Their GitHub Projects? In: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR); 2019. p. 235-9.
48. Bangash AA, Sahar H, Chowdhury S, Wong AW, Hindle A, Ali K. What do Developers Know About Machine Learning: A Study of ML Discussions on StackOverflow. In: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR); 2019. p. 260-4.
49. Nugroho YS, Islam S, Gunawan D, Kurniawan YI, Hossain MJ. Dataset of network simulator related-question posts in stack overflow. Data in Brief. 2022;41:107942. doi:<https://doi.org/10.1016/j.dib.2022.107942>.
50. Dabic O, Aghajani E, Bavota G. Sampling Projects in GitHub for MSR Studies. In: Proceedings of the 18th International Conference on Mining Software Repositories (MSR '21); 2021. p. 560-4.
51. Cass S. The Top Programming Languages 2024; 2024. <https://spectrum.ieee.org/top-programming-languages-2024>.
52. Zakeri-Nasrabadi M, Parsa S, Ramezani M, Roy C, Ekhtiarzadeh M. A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. Journal of Systems and Software. 2023 10;204:111796. doi:10.1016/j.jss.2023.111796.
53. Sajnani H, Saini V, Svajlenko J, Roy CK, Lopes CV. SourcererCC: Scaling Code Clone Detection to Big-Code. In: Proceedings of the 38th International Conference on Software Engineering (ICSE '16). ACM Press; 2016. p. 1157-68.
54. Ragkhitwetsagul C, Krinke J, Clark D. A comparison of code similarity analysers. Empirical Software Engineering. 2018 August;23(4):2464-519.
55. Ragkhitwetsagul C, Krinke J, Paixao M, Bianco G, Oliveto R. Toxic Code Snippets on Stack Overflow. IEEE Transactions on Software Engineering. 2021;47(3):560-81.
56. Ragkhitwetsagul C, Krinke J, Paixao M, Bianco G, Oliveto R. Toxic Code Snippets on Stack Overflow. IEEE Transactions on Software Engineering. 2021;47(3):560-81. doi:10.1109/TSE.2019.2900307.
57. Tempero E, Anslow C, Dietrich J, Han T, Li J, Lumpe M, et al. The qualitas corpus: A curated collection of java code for empirical studies. In: 2010 Asia pacific software engineering conference. IEEE; 2010. p. 336-45.
58. Wang T, Harman M, Jia Y, Krinke J. Searching for better configurations: a rigorous approach to clone evaluation. In: FSE '13; 2013. p. 455-65.
59. Ragkhitwetsagul C, Paixao M, Adham M, Busari S, Krinke J, Drake JH. Searching for configurations in clone evaluation—a replication study. In: SSBSE '16; 2016. p. 250-6.
60. Zampetti F, Bavota G, Canfora G, Penta MD. A Study on the Interplay between Pull Request Review and Continuous Integration Builds. In: Proceedings of the 26th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '19); 2019. p. 38-48.
61. Gonzalez D, Zimmermann T, Nagappan N. The State of the ML-universe. In: Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20). ACM; 2020. p. 431-42.

62. Sheoran J, Blincoe K, Kalliamvakou E, Damian D, Ell J. Understanding "watchers" on GitHub. In: Proceedings of the 14th International Conference on Mining Software Repositories (MSR '14); 2014. p. 336-9.
63. Munaiah N, Kroh S, Cabrey C, Nagappan M. Curating GitHub for engineered software projects. *Empirical Software Engineering*. 2017 Dec;22(6):3219-53.
64. Shapiro SS, Wilk MB. An analysis of variance test for normality (complete samples). *Biometrika*. 1965;52(3-4):591-611.
65. Kruskal WH, Wallis WA. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*. 1952;47(260):583-621.
66. Williams D, Callan J, Kirbas S, Mechtaev S, Petke J, Prideaux-Ghee T, et al. User-Centric Deployment of Automated Program Repair at Bloomberg. In: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP '24. New York, NY, USA: Association for Computing Machinery; 2024. p. 81-91. doi:10.1145/3639477.3639756.
67. Watanabe M, Kula RG, Fujino S, Uchida S, Fowze F, Zhu H, et al. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *ACM Transactions on Software Engineering and Methodology*. 2025.
68. Pinto G, Steinmacher I, Gerosa MA. Who Gets a Patch Accepted First? Comparing the Contributions of Employees and Volunteers. In: Proceedings of the 11th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE); 2018. p. 110-3. doi:10.1145/3195836.3195858.
69. Liu Y, Le-Cong T, Widyasari R, Tantithamthavorn C, Li L, Le XBD, et al. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology*. 2024 6;33:1-26. doi:10.1145/3643674.
70. Wang J, Luo X, Cao L, He H, Huang H, Xie J, et al.. Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval; 2024. Available from: <https://arxiv.org/abs/2407.02395>. arXiv:2407.02395.
71. Fu Y, Liang P, Tahir A, Li Z, Shahin M, Yu J, et al. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Trans Softw Eng Methodol*. 2025 Oct;34(8). doi:10.1145/3716848.
72. Siddiq ML, Roney L, Zhang J, Santos JCDS. Quality Assessment of ChatGPT Generated Code and their Use by Developers. In: Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24). New York, NY, USA; 2024. p. 152-156. doi:10.1145/3643991.3645071.
73. Dou S, Jia H, Wu S, Zheng H, Zhou W, Wu M, et al.. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study; 2024. Available from: <https://arxiv.org/abs/2407.06153>. arXiv:2407.06153.
74. Tamboon F, Moradi-Dakhel A, Nikanjam A, Khomh F, Desmarais MC, Antoniol G. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering*. 2025 feb;30(3):65. doi:10.1007/s10664-025-10614-4.
75. Cotroneo D, Improta C, Liguori P. Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity; 2025. Available from: <https://arxiv.org/abs/2508.21634>. arXiv:2508.21634.

76. Verdi M, Sami A, Akhondali J, Khomh F, Uddin G, Motlagh AK. An Empirical Study of C++ Vulnerabilities in Crowd-Sourced Code Examples. IEEE Transactions on Software Engineering. 2022;48(5):1497-514. doi:10.1109/TSE.2020.3023664.

Appendices


```

1 @@ -93,16 +93,21 @@ private final static String escapeEnding(
   String s) {
2 // File utils
3 public final static void copy(File src, File dst) throws
   IOException {
4     InputStream in = new FileInputStream(src);
5 -     OutputStream out = new FileOutputStream(dst);
6 -
7 -     // Transfer bytes from in to out
8 -     byte[] buf = new byte[1024];
9 -     int len;
10 -    while ((len = in.read(buf)) > 0) {
11 -        out.write(buf, 0, len);
12 -    }
13 -    in.close();
14 -    out.close();
15 +    try {
16 +        OutputStream out = new FileOutputStream(dst);
17 +        try {
18 +            // Transfer bytes from in to out
19 +            byte[] buf = new byte[1024];
20 +            int len;
21 +            while ((len = in.read(buf)) > 0) {
22 +                out.write(buf, 0, len);
23 +            }
24 +        } finally {
25 +            out.close();
26 +        }
27 +    } finally {
28 +        in.close();
29 +    }
30 }
31

```

Fig 14. Bug-fix recommendation in the cyriux project pull request, ensuring proper closing of the OutputStream out (<https://github.com/cyriux/mpcmaid/pull/16>).

```

1 @@ -6,6 +6,9 @@
2 public class AddSubtractDaysSkippingWeekendsUtils {
3
4     public static LocalDate addDaysSkippingWeekends(LocalDate date
5     , int days) {
6         +     if (days < 1) {
7         +         return date;
8         +     }
9         LocalDate result = date;
10        int addedDays = 0;
11        while (addedDays < days) {
12 @@ -18,6 +21,9 @@ public static LocalDate addDaysSkippingWeekends(
13        LocalDate date, int days) {
14        }
15
16        public static LocalDate subtractDaysSkippingWeekends(LocalDate
17        date, int days) {
18        +     if (days < 1) {
19        +         return date;
20        +     }
21        LocalDate result = date;
22        int subtractedDays = 0;
23        while (subtractedDays < days) {

```

Fig 15. Bug-fix recommendation in the eugenp project pull request, checking the invalid days value (<https://github.com/eugenp/tutorials/pull/18370>).